

Diseño de Máquinas de Estado en Verilog

Electrónica Digital

May 19, 2026

¿Qué es una Máquina de Estados?

Una FSM (Finite State Machine) es un sistema secuencial cuyo comportamiento depende de:

- Estado actual.
- Entradas del sistema.
- Reglas de transición.

Las FSMs son utilizadas en:

- Procesadores.
- Protocolos de comunicación.
- Controladores industriales.
- Robots.
- Sistemas embebidos.

Elementos Fundamentales de una FSM

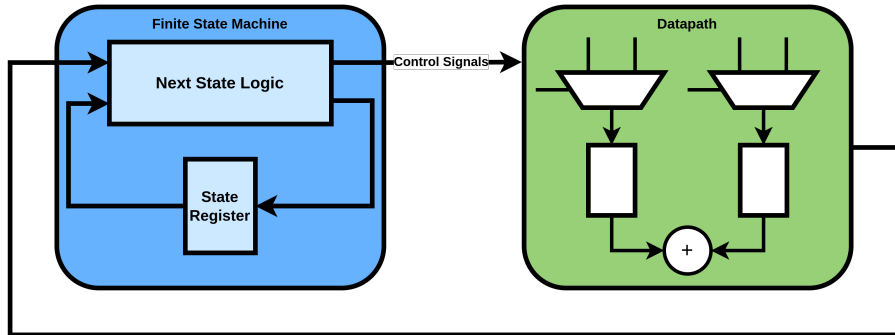


Figure: FSM y Datapath

La FSM contiene:

- Registro de estado.
- Lógica de transición.
- Lógica de salida.

Tipos de FSM

Máquina de Moore

Las salidas dependen únicamente del estado actual.

$$Salida = f(Estado)$$

Máquina de Mealy

Las salidas dependen del estado y de las entradas.

$$Salida = f(Estado, Entrada)$$

Moore

- Más estable.
- Salidas sincronizadas.
- Menos glitches.
- Más usada en FPGA.

Mealy

- Respuesta más rápida.
- Menos estados.
- Más sensible al ruido.
- Puede generar glitches.

- 1 Definir entradas y salidas.
- 2 Dividir el comportamiento en estados.
- 3 Dibujar el diagrama de estados.
- 4 Construir tabla de transición.
- 5 Implementar en HDL.
- 6 Simular.
- 7 Sintetizar.

Diseñar un sistema automático de acceso a un parqueadero.

El sistema debe:

- 1 Esperar un vehículo.
- 2 Leer una secuencia serial.
- 3 Verificar identificación.
- 4 Abrir barrera.
- 5 Esperar el paso del vehículo.
- 6 Cerrar barrera.

Entradas

- clk
- rst
- sensor_vehiculo
- bit_serial
- dato_listo
- sensor_salida

Salidas

- shift_enable
- comparar_id
- servo_abrir
- servo_cerrar

Dividir el sistema en estados simples.

- 1 **IDLE**: espera de vehículo.
- 2 **LEER**: captura de datos seriales.
- 3 **VALIDAR**: comparación de identificación.
- 4 **ABRIR**: activación de la barrera.
- 5 **ESPERAR**: espera de paso del vehículo.
- 6 **CERRAR**: cierre de la barrera.

Tabla de Transiciones

Estado Actual	Condición	Siguiente Estado
IDLE	sensor_vehiculo = 1	LEER
LEER	dato_listo = 1	VALIDAR
VALIDAR	id_correcto = 1	ABRIR
VALIDAR	id_correcto = 0	IDLE
ABRIR	timer_done = 1	ESPERAR
ESPERAR	sensor_salida = 1	CERRAR
CERRAR	timer_done = 1	IDLE

¿Cómo leer datos seriales?

Usamos un Shift Register:

$$Q \leftarrow \{ Q[n - 2 : 0], \textit{bit_serial} \}$$

Cada pulso de reloj desplaza un nuevo bit.

Registro de Desplazamiento en Verilog

```
1 module shift_register(  
2     input clk,  
3     input rst,  
4     input enable,  
5     input serial_in,  
6     output reg [7:0] data  
7 );  
8 always @(posedge clk) begin  
9     if(rst)  
10         data <= 8'b0;  
11     else if(enable)  
12         data <= {data[6:0], serial_in};  
13 end  
14 endmodule
```

Codificación de Estados

```
1 localparam IDLE      = 3'b000;
2 localparam LEER       = 3'b001;
3 localparam VALIDAR    = 3'b010;
4 localparam ABRIR      = 3'b011;
5 localparam ESPERAR    = 3'b100;
6 localparam CERRAR     = 3'b101;
```

Registro de Estado

```
1 reg [2:0] estado_actual;  
2 reg [2:0] siguiente_estado;  
3  
4 always @(posedge clk) begin  
5     if(rst)  
6         estado_actual <= IDLE;  
7     else  
8         estado_actual <= siguiente_estado;  
9 end
```

Lógica de Transición

```
1 always @(*) begin
2     case(estado_actual)
3         IDLE:
4             if(sensor_vehiculo)
5                 siguiente_estado = LEER;
6             else
7                 siguiente_estado = IDLE;
8         LEER:
9             if(dato_listo)
10                 siguiente_estado = VALIDAR;
11             else
12                 siguiente_estado = LEER;
13         VALIDAR:
14             if(id_correcto)
15                 siguiente_estado = ABRIR;
16             else
17                 siguiente_estado = IDLE;
18     endcase
19 end
```

```
1 always @(*) begin
2
3     shift_enable = 0;
4     servo_abrir = 0;
5     servo_cerrar = 0;
6
7     case(estado_actual)
8
9         LEER:
10             shift_enable = 1;
11
12         ABRIR:
13             servo_abrir = 1;
14
15         CERRAR:
16             servo_cerrar = 1;
17
18     endcase
19
```

- No definir estado por defecto.
- Crear loops combinacionales.
- Mezclar lógica secuencial y combinacional.
- No sincronizar entradas externas.
- Usar clocks derivados incorrectamente.

Problema:

Las entradas externas NO están sincronizadas al reloj.

Solución:

- Sincronizadores.
- Debouncing.
- Detectores de flanco.

FSM + Shift Register + Comparador + Servo

Cada módulo tiene una función específica.

La FSM coordina TODO el sistema.

Ahora ustedes deben diseñar:

- Máquina de estados.
- Registro de desplazamiento.
- Comparación de contraseña.
- Control del servo.
- Manejo de teclado.

La metodología es EXACTAMENTE la misma.

- Un módulo por funcionalidad.
- FSMs simples.
- Nombres claros.
- Simular antes de sintetizar.
- Usar comentarios.
- Verificar timing.

- Las FSMs son fundamentales en diseño digital.
- Permiten controlar sistemas complejos.
- Verilog facilita la implementación hardware.
- El diseño modular simplifica depuración.

Preguntas